Mundarija

#1 Kirish

Python mashhur dasturlash tilidir. U Guido van Rossum tomonidan yaratilgan va 1991 - yilda chiqarilgan.

Python yuqori darajali, umumiy maqsadli va juda mashhur dasturlash tilidir. Python dasturlash tili (so'nggi Python 3) dasturiy ta'minot sanoatidagi barcha ilg'or texnologiyalar bilan bir qatorda veb-saytlar ishlab chiqish, Machine Learning ilovalarida qo'llaniladi. Python dasturlash tili yangi boshlanuvchilar uchun, shuningdek C++ va Java kabi boshqa dasturlash tillariga ega bo'lgan tajribali dasturchilar uchun juda mos keladi.

Quyida Python dasturlash tili haqida ba'zi faktlar keltirilgan:

Python hozirda eng ko'p qo'llaniladigan ko'p maqsadli, yuqori darajali dasturlash tilidir. Python ob'ektga yo'naltirilgan va protsessual paradigmalarda dasturlash imkonini beradi. Python dasturlari odatda Java kabi boshqa dasturlash tillariga qaraganda kichikroqdir. Dasturchilar tilning nisbatan kamroq kod yozishlari kerak, bu ularni doimo o'qilishi osonlashtiradi.Python tili deyarli barcha texnologik gigantlar tomonidan qo'llaniladi - Google, Amazon, Facebook, Instagram, Dropbox, Uber va boshqalar.Pythonning eng katta kuchi quyidagilar uchun ishlatilishi mumkin bo'lgan standart kutubxonaning ulkan to'plamidir:

- Mashina o'rganish (Machine Learning)
- GUI ilovalari (masalan, Kivy , Tkinter, PyQt va boshqalar)
- Django,Flask kabi veb-ramkalar (YouTube, Instagram, Dropbox tomonidan qo'llaniladi)
- Rasmga ishlov berish (masalan, OpenCV , Pillow)
- Veb-skarping (masalan, Scrapy, BeautifulSoup, Selenium)
- Sinov ramkalari
- Multimedia
- Ilmiy hisoblash
- Ma’lumotlarni qayta ishlash va boshqalar..

Python bu umumiy maqsadli dasturlash uchun keng tarzda foydalaniladigan yuqori darajali dasturlash tili, chunki o'rganish oson va qulay sintaksisga ega. Undan tashqari skriptli dasturlash tillariga kiradi. Python dinamik tipizatsiyaga ega, obyektga yo'naltirilgan dasturlash, funksional dasturlash, strukturali, avtomatik xotirani boshqarish va albatta ko'p patokli dasturlash tillaridan biri. Python har xil platformalar uchun yozilgan masalan Windows, Linux, Mac OSX, Palm OS, Mac OS va hokazo. Python Microsoft.NET platformasi uchun yozilgan realizatsiyasi ham bor uni nomi IronPython. Bugungi kunda dunyoga mashhur ko`plab kompaniyalar NASA, Google, Yandex, CERN, Apple computer, Dream Works, kosmik teleskop institutlari Pythonni ishlatishadi.

Python boshqa dastulash tiliga qaraganda ancha oson va kamroq kod yozish imkonini beradi masalan:

Hello world dasturni ishga tushirish uchun dasturlash tillarida kod quyidagicha yoziladi:

Java dasturlash tilida

```
public class HelloWorld
{
   public static void main (String[] args)
   {
      System.out.println("Hello, world!");
   }
}
```

C dasturlash tilida

```
#include <stdio.h>

int main() {
      printf(“Hello, world!”);
      return 0;
}
```

Python dasturlash tilida

```
print(“Hello, world!”)
```

Rasmiy web sayti https://python.org hisoblanadi

Dasturlash tili haqida umumiy ma’lumotlarni ushbu saytning Documentation bo’limidan olishingiz mumkin.Undan tashqari bu sayt orqali siz Dasturlash tili imkoniyatlari, ushbu dasturlash tili haqida yangiliklarni kuzatib borishingiz mumkin.Dasturlashni chuqur bilgaanlar uchun vakansiyalar ham topilishi mumkin

#2 Pythonni o’rnatish

Python interpretatorini komputerga o’rnatish uchun rasmiy saytga tashrif buyuramiz

https://python.org

va Downloads menusidan eng so’ngi chiqqan versiyasini yuklab olamiz va sichqonchaning chap tugmasini ikki marta bosib o’rnatishni boshlaymiz.

#3 PyCharm dasturni o’rnatish

Pycharm dasturni biz https://jetbrains.com/pycharm sahifasiga web brouzer orqali kiamiz va quyidagi sahifa chiqadi.

Va biz bu sahifadan “Download” buyrug’ini tanlaymiz va yana quyidagi sahifa ochiladi.

Bu yerdan biz PyCharm dasturining 2 xil versiyasi chiqadi ‘Professional’ va ‘Community’ bundan biz 2-versiyasini ya’ni “Community” versiyasini tanlab yuklab olamiz va uni ham kompyterimizga o’rnatamiz chunki bu dasturning tekin versiyasi hisoblanadi va dasturga boyagi Pythonning 3.11.1 versiyasining interpretatorini Pycharm dasturiga bog’laymiz va u quyidagicha bo’ladi.

PyCharm dasturiga kiramiz va “File” menusiga kiramiz File menusidan esa ‘Settings’ buyrug’ini tanlaymiz va yana quyidagi oyna hosil buladi.

Project menusidan ‘Python Interpreter’ buyrug’ini tanlaymiz quyidagi oyna hosil bulishi uchun:

Add Interpreter buyrug’i orqali Pythonning 3.11.1 versiyasining Interpretatorini qo’shamiz yoki bulmasa avomatik ravishda dastur Python Interpretatorini uzi izlab topadi.

#4 Python dasturlash tili sintaksisi

Python sintaksisi juda soda va oson bundan tashqari juda kam kod yozish imkonini beradi.Python dasturlash tilida kodlarni yozish boshqa dastulash tiliga qaraganda juda kam kod yozish mumkin.U dan tashqari boshqa dasturlash tillaridagi kabi kod oxiriga nuqali vergul quyish talab qilinmaydi(;).

Masalan salom dunyo(Hello, world) dasturini ishga tushirish quyidagicha buladi:

```
print(“Hello, world!”)
```

biror bir funksiyani boshqa aksariyat dasturlash tilida yozish uchun quyidagicha yozishimiz mumkin:

masalan C dasturlash tilida ikki sonni yig’indisini hisoblovchi dastur tuzamiz

```
#include <stdio.h>

int summa(int a,int b) {

  printf("Yig'indi %d",a+b);

}
```

```
int main() {
summa(4,6);
   return 0;
}
```

Pyhton dasturida esa quyidagicha:

```
def Summa(a,b):
  print(f“Yig’indi {a+b}”)
Summa(4,5) #9
```

Undan tashqari Daturlash tilda kommentariya (izohlar) quyidagicha yoziladi

```
# belgisi orqali bir qatorli izohlar ,
‘’’
Ko’p qatorli
Izohlar
‘’’
```

Yaratgan python dasturimizni quyidagicha ishga tushirishimiz mumkin:
Pusk + R tugmasini bosib cmd buyrug’ini beramiz.

Va quyidagicha oyna hosil buladi oynada biz daturimizni ishga tushirishimiz mumkin

Bu yerda python_syntax.py dasturimiz fayli hisoblanadi

Dastur fayllarining formati .py , .pyw , pyi formatda bo’lishi mumkin

#5 Pythonda Son.Arifmetik Amallar

Python dasturlash tilida arifmetik amallar huddi oddiy hisob - kitobdagidek bir xil.

Sonlar ham bir xil hisoblanadi.Python dasturida 3 xil asosiy son turi mavjud.

Bular `int, float` va `complex` sonlardir.Bu sonlarni dasturda kurish uchun quyidagicha yozib kurishimiz mumkin.

```
print(type(5))       # <class 'int'>
print(type(1.25))    # <class 'float'>
print(type(1j))      # <class 'complex'>
```

bu sonlarni bir biriga konvertatsiya qilish ham mumkin masalan 5 sonini float tipiga olib kelish mumkin yoki complex qiymatga olib kelish mumkin va esa aksincha ham bulishi ham mumkin masalan:

```
a = 5
print(float(a))  #5.0
print(complex(a))  #(5 + 0j)
```

Python dasturlash tilida sonlar ustida bemalol arifmetik amallarni bajarsa buladi va ular Python dasturlash tilida asosiy 7 ta hisoblanadi

+	Qo'shish operatori	`print(5+2)  #7`
-	Ayirish operatori	`print(5-2)  #3`
*	Ko'paytirish operatori	`print(5*2)  #10`
/	Bo'lish operatori	`print(5/2)  #2.5`
%	Qoldiq operatori	`print(5/%2)  #1`
**	Darajaga ko'tarish operatori	`print(5**2)  #25`
//	Bo'luvchining butun qismini olish operatori	`print(5//2)  #2`

Pythonda ham amallar ustuvorligi mavjud masalan:

```
print(2 + 2 * 5)  # 12
print(2 – 5 / 2)  # -0.5 va buning turi float bulib chiqyapti
print(25 ** 0.5)  # 5.0 buning ham ma'lumot turi float hisoblanadi
print(0.1 + 0.2)  # 0.30000000000000004 mana shunday qiymat chiqadi
```

buni soddalashtirish quyidagicha bo'ladi

```
print('%.2f' % (0.1+0.2))  # 0.30
```

#6 O’zgaruvchilar

Ma’lumotlar ustida ishlash ya’ni uni tahrirlash, ko’chirish,o’chirish va hokozo amallarni bajarish uchun unga biror bir o’zgaruvchi berishimiz kerak.O’zgaruvchilarni qiymati har xil bo’lishi mumkin va uni kiritayotgan vaqtimiz boshqa dasturlash tillari kabi ma’lumot turini kiritish talab qilinmaydi.O’zgaruvchilarning o’zgarmaslardan (Konstanta) farqi nomidan ham bilinib turibdike uning ustida ishlab ma’lumot turlarini o’zgartirishimiz mumkin o’zgarmaslar esa bir ma’lumot beriladi keyin uni o’zgartirib bulmaydi.

```
x = 5 #bu o’zgaruvchining ma’lumot turi int turiga tegishli

print(x) #5

print(type(x)) #<class 'int'>
```

Endi o’zgaruvchini qiymatini yoki ma’lumot turini o’zgartirib kuramiz yani butun son turidan satr ko’rinishiga o’tkazamiz

```
x = 5

print(x, type(x))  # 5 <class ‘int’>

x = ‘dastur’

print(x, type(x))  # dastur <class ‘string’>
```

o’zgaruvchilarni nomlashda o’zgaruvchi nomlari harf yoki tag chiziqcha (_) bilan boshlanishi kerak aks xolda o’zgaruvchi nomi xato kiritilgan bo’ladi.

```
Myname = ‘John Doe’
```

```
myname2 = ‘John Doe’

_myname = ‘John Doe’

my_name = ‘John Doe’
```

O’zgaruvchi nomini raqam bilan boshlash yoki boshqa bir belgili simvollar va probellar bilan ifodalsh mumkin emas

~~`my-name = ‘John Doe’`~~

~~`my name = ‘John Doe’`~~

~~`2myname = ‘John Doe’`~~

Bir nechta o’zgaruvchilarga qiymat berish ham mumkin masalan

```
a, b, c = “Apple”, “Banana”, “Orange”

print(a) # Apple

print(a, b, c) # Apple Banana Orange
```

Bir necha o’zgaruvchini ham bir xil qiymat berish ham mumkin:

```
x, y, z = “Fruits”

print(y)  # Fruits
```

O’zgaruvchilar ustida amallar ham qilish mumkin masalan “+” qo’shish belgisi yordamida son qiymatlarini qo’shish mumkin bo’lsa, satr ma’lumotlarini esa biriktirish mumkin

```
a = ‘Hello’

b = ‘World’

print(a + b)   #HelloWorld

x = 5

print(a + x)   # xatolik chiqaradi
```

takroriy yozish uchun esa mana bunday qilishimiz mumkin

```
print(a * x)    # HelloHelloHelloHelloHello
```

O’zgaruvchilar 2 xil buladi local va global o’zgaruvchilar hisoblanadi masalan

```
x = “SALOM”   ==>>  Bu Global o’zgaruvchi hisoblanadi

def Func():

    x = “Hello”  ==>> Bu esa local o’zgaruvchi hisoblanib faqat
    funksiya ichidagina ishlaydi
```

```
    print(x + " World")

Func()
print(x + " World")

Hello World
SALOM World
```

Biror bir funksiya ichidagi o'zgaruvchini ham global o'zgaruvchi sifatida qabul qilish mumkin buning uchun global kalit so'zidan foydalanamiz

```
def Func():
    global x
    x = "HELLO"
    print(x + " WORLD")
Func() # HELLO WORLD
print(x) # HELLO
```

#7 Mantiqiy operator va elementlar

Pythonda mantiqiy operatorlar `and, or, not` kabi mantiqiy operatorlar mavjud:

and – agar ikkala ifoda rost busa rost,qiymat qaytaradi
or – agar ikkala ifodadan biri rost busa rost qiymat qaytaradi
not – sahrtga muvofiq teskari qiymat qaytaradi , rost busa yolg'on , yolg'on busa rost qiymat chiqaradi

Buning natijalari True (Rost) va yoki Fale (Yolg'on) qiymat chiqarishi mumkin
Masalan:

```
a = 5
print(6>a and a>3)  #True
print(4>a and a>2)  #False
print(4>a or a>1)   #True
print(not(a>3 and a<2)) #True
my_true = True
my_false = False
print(my_true, type(my_true)) #True <class 'bool'>
```

#8 Turli xil operatorlar

Python dasturlash tilida operatorlar quyidagilar kiradi:

- Arifmetik operatorlar
- O’zlashtirish operatorlari
- Taqqoslash operatorlari
- Mantiqiy operatorlar
- Aniqlash operatorlari
- A’zolik operatorlari
- Bitli operatorlar

Bu yerdan biz arifmetik operatorlar hamda mantiqiy oeratorlarni siz bilan o’tgan darslarimizda ko’rib o’tdik bugun esa qolganini o’tamiz

O’zlashtirish operatorlari

=	**x = 5**	**x=5**
+=	**x += 3**	**x = x + 3**
- =	**x -= 3**	**x= x - 3**
***=**	**x *= 3**	**x= x * 3**
/=	**x /= 5**	**x = x / 5**
%=	**x %= 5**	**x = x % 5**
//=	**x //= 5**	**x = x // 5**
****=**	**x **= 5**	**x = x ** 3**

&= **x &= 3** **x = x & 3**

|= **x |= 3** **x = x |3**

^= **x ^= 3** **x = x ^ 3**

>>= **x >>= 3** **x = x >> 3**

<<= **x <<= 3** **x = x << 3**

Taqqoslash operatorlari

= = **Teng** **x == y**

!= **Teng emas** **x != y**

> **Katta** **x > y**

< **Kichik** **x < y**

>= **Katta yoki teng** **x >= y**

<= **Kichik yoki teng** **x <= y**

Aniqlash operatorlari

Aniqlash operatorlari ikkala obyektni solishtirishda foydalaniladi.Ularni qiymati emas balki ular bir xil ekanligi tekshiriladi.Aniqlash operatorlari 2 xil bo'ladi.

- is - Ikkala o'zgaruvchi bir xil busa rost aks holda yolg'on qiymat hosil qiladi
- is not - Obyektlar bir xil busa yolg'on , har xil busa rost qiymat qabul qiladi

```
x = ["apple", "banana"]
y = ["apple", "banana"]
z = x
print(x is z)
print(x is y)
print(x == z)
print(x is not z)
print(x is not y)
print(x != z)

#------------------------------------------------------------

True
False
True
False
True
False
```

A'zolik operatorlari

A’zolik operatori biror bir qiymat obyekt ichiga tegishli ekanligiga tegishli yoki tegishli emasligini tekshiradi

`in` - Belgilangan qiymat obyektda mavjud bo’lsa, rost qiymat qaytaradi.
`not in` - Belgilangan qiymat obyektda mavjud bo’lmasa, rost qiymat qaytaradi.

```
x = ["audi", "mustang"]

print("audi" in x)

print("audi" not in x)

True
False
```

Bitli Operator mustaqil o’rganish uchun vazifa

#9 Satrlar

Python dasturlash tilida satrlar bilan ishlash juda qulay va samarali hisoblanadi.Satr deb , simvollardan tashkil topgan ketma-ketlikka satr deb ataladi.Pythonda satrlar qo’shtirnoq yoki bittalik tirnoq ichiga oli yoziladi va uni keyin ekranga chiqarish mumkin.Satrlar ustida uslublar , ularni o’zgartirish va hakozo amallarni qilish mumkin.

```
words = “Hello, world!”

print(words)  # Hello, world!
```

va yoki bittalik tirnoqqa olib ham yozish mumkin.

```
words = ‘Hello, world!’

print(words)  # Hello, world!
```

masalan qo’shtirnoq ichida qo’shtirnoq ishlatib bulmaydi

`print(“Hello, “world”!”)` deb yozib bumaydi

xatolik chiqaradi shuning uchun unga **(\)** mana bu belgi orqali yozish mumkin

```
print(“Hello, \”world\”!”)  #Hello, “world”!
```

ko’p qatorli satr hosil qilish uchun esa

```
abs = ‘’’Bu ko’p

qatorli satr
```

```
hosil qilish’’’

print(abs)    #Bu ko’p

               qatorli satr

               hosil qilish
```

masalan ikkinchi yo’li ham bor

```
abs = ‘Hello world\n\

Hello world\n\

Hello world’

print(abs)   # Hello world

               Hello world

               Hello world
```

#10 Satrlar ustida operatsiyalar

Masalan fayl yo’lini ko’rsatish uchun **C:\Program Files\t\n\text.txt** ni ifodalashda xuddi uzidek kiritishda xatolik yuz beradi

```
print('C:\root\sys\t\text.txt')  # C:\oot\sys       ext.txt
```

ushbuni ekranga chiqaradi albatta bu xato.Buni tug’rilash uchun “\” belgisidan foydalanamiz

```
print('C:\\root\sys\\t\\text.t')     # C:\root\sys\t\text.t
```

Buning uchun quydagi jadvaldan asosiylarini o’rganamiz

\n - yangi qatorga o’tish

\t - Gorizantal tabulyatsiya

\’ - tutiq belgisini

\xhh - belgini 16lik sistemasiga o’tkazish

\ooo - belgini 8lik sistemasiga o’tkazish

\\ - **backslash belgisi**

qolgani uyga vazifa

undan tashqari

```
s = 'py''thon'
print(s)  # python
```

Konkatenatsiya

```
s1 = ‘Hello’
s2 = ‘ world’
s3 = s1 + s2
print(s3)  #Hello world
```

Masalan:

```
name ='John'
age = 32
print('My name is ' + name + ".I'm " + str(age))
# My name is John.I'm 32

print('h1h1h1')   <<===>>  print(‘h1’*3)
h1h1h1
```

Satrlar aslida ro’yxatlardan iborat bo’ladi va satrning birinchi elementi 0 element bo’ladi. Masalan

```
s = “Hello world”
print(s[0])  #H
print(s[x:y:z])
print(s[start:stop:step])
```

bu yerda x – satrni boshlanish elementi
y – satrni tugash elementi
z – satrni qadamlash

```
Hello world
012345678910
```

probel ham element soniga kiradi

`>>print(s[0:11])` bu **Hello world** o’zgarmay chiqadi xhunki 0 elementdan boshlanib 11 elementda tugayapti. 11 - element javobga kirmaydi.

`>>print(s[0:5])` bu yerda **Hello** oynaga chiqadi 0 elementdan 4 elementgacha 5 ta element

`>>print(s[6:])` bu yerda **world** chiqadi ya’ni 6 elementdan oxirgi elementgacha

`>>print(s[::2])` bu yerda **Hlowrd** chiqadi ya’ni satrning juft indeksli elementlari

>>print(s[::-1]) bu yerda **dlrow olleH** satri chiqadi yani minus quyish orqadan boshlab satrni ifodalaydi

```
>>print(s[:5] + s[6:]) # Helloworld
>>print(s[:5], s[6:])  #Hello world
```

#11 Satr metodlar

Pythonda satr metodlari doimo funksiya shaklida bo'ladi.Pythonda hamma satr metodlarini bilish uchun dir() funksiyasini ishlatish bilan bo'ladi.

```
x = dir(str)
```

print(x) #list shaklida chiqadi ya'ni

```
# ['__add__', '__class__', '__contains__', '__delattr__',
'__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__getitem__', '__getnewargs__',
'__getstate__', '__gt__', '__hash__', '__init__',
'__init_subclass__', '__iter__', '__le__', '__len__', '__lt__',
'__mod__', '__mul__', '__ne__', '__new__', '__reduce__',
'__reduce_ex__', '__repr__', '__rmod__', '__rmul__', '__setattr__',
'__sizeof__', '__str__', '__subclasshook__', 'capitalize',
'casefold', 'center', 'count', 'encode', 'endswith', 'expandtabs',
'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha',
'isascii', 'isdecimal', 'isdigit', 'isidentifier', 'islower',
'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper',
'join', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition',
'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex',
'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines',
```

```
'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper',
'zfill']
```

```
print(len(x)) #81 ya’ni 81 ta satr metodlari bor
```

biz siz bilan asosiylarini ko’rib chiqamiz

```
s = "Welcome TO my WoRld"

s = s.capitalize()

print(s)  # Welcome to my world

s = s.count(‘l’) #5 bu funksiya satrda l harfi nechanchi
elementligini kursatadi

s = s.find(‘l’) #2 satrdan topish va nechanchi o’rindaligini
chiqaradi

s = s.index(‘l’) #2 bu ham izlaydi faqat bulmasa xatolik chiqaradi

s = s.isalpha() #False agar satrda probel busa xatolik chiqaradi

s = s.isdigit() #False agar satr barcha qiymati raqamdan tashkil
topgan busa True deb chiqaradi

s = ‘welcome to my world’

s = ‘.’.join() #w.e.l.c.o.m.e. .t.o. .m.y. .w.o.r.l.d chiqadi
orasiga qushish

s = s.replace(‘world’, ‘office’) #welcome to my office

s = s.split() #['welcome', 'to', 'my', 'world'] satrni
list(ro’yxat) ko’rinishiga o’tkazadi

r = ‘My name is Shaxboz.\nI am 36 years old’

r = r.splitlines(True)

print(r)  #[‘My name is Shaxboz.\n’, ‘I am 36 years old’]

s = ‘welcome to my world’

s = s.title()

print(s) #Welcome To My World

s = s.upper() #WELCOME TO MY WORLD

s = s.lower() #welcome to my world
```

#12 Satrlarni formatlash

Pythonda satrlarni formatlash quyidagi format() funksiyasi yordamida amalga oshiramiz.Funksiyaning umumiy ko'rinishi quyidagicha

string.format(value1,value2,value3……)

```
txt = "The price of the car is {} dollars"
print(txt.format(4000)) # The price of the car is 4000 dollars
```

satrlarni formatlashni 3 ta asosiy ko'rinishi mavjud

```
1.txt = “My name is {name}.I am {age} years old”
print(txt.format(name='John', age=36)) # My name is John.I am 32 years old
2.txt = “My name is {0}.I am {1} years old”
print(txt.format('John', 32)) # My name is John.I am 32 years old
3.txt = “My name is {}.I am {} years old”
print(txt.format('John', 32)) # My name is John.I am 32 years old
```

Undan tashqari

```
name = ‘John’
age = 32
print(f“My name is {name}.I am {age} years old”)
# My name is John. I'm 32 years old


print(f“5 + 2 = {5 + 2}”) # 5 + 2 = 7
```

Format turlari uyga vazifa mustaqil o’rganish uchun

#13 IF operatori

Pythonda shartli operator sifatida if nomli operator ishlatiladi.Uning umumiy ko’rinishi quyidagicha

```
if 1-ifoda:
    1-operator
elif 2-ifoda:
    2-operator
else:
    3-operator
```

Shartli operator bajarilayotganda birinchi ifoda hisoblaniladi.Agar ifoda rost busa, ya’ni noldan farqli busa 1-operator bajariladi.Agar 1-ifoda yolg’on bulsa 2 – ifoda bajariladi agar u noldan farqli yani rost qiymat qabul qilsa 2- operator bajariladi.U ham yolg’on bulsa else yani 3- operator bajariladi.

```
x = 0
if x:
    print(“To’g’ri”)
else:
    print(“Noto’g’ri”)
# Noto’g’ri
```

svetafor dasturni tuzish

```
light = ‘red’
if light == ‘red’:
     print(“Stop”)
elif light == ‘yellow’:
     print(“Wait”)
elif light == ‘green’:
     print(“Go”)
else:
     print(“What ?”)
```

yana bir qiziq dastur:

```
age = int(input(“Yoshingiz nechada: ”))
if age >= 18:
     print(“Xush kelibsiz”)
else:
     print(“Mumkin emas”)
     print(f“Sizni yoshingiz {age} da.Yana {18 - age} yil kerak”)
```

qisqacha

```
x = 1
print(“Ok” if x else “No”) #ok
```

misol kiritilgan 3 ta sonni kattasini topish dasturini tuzamiz

```
a = int(input("a sonini kiriting"))
b = int(input("b sonini kiriting"))
c = int(input("c sonini kiriting"))

if a>b:
    if a>c:
        maxim = a
    else:
        maxim = c
else:
    if b>c:
        maxim = b
    else:
        maxim = c
```

```
print(f"Eng kattasi {maxim}")
```

#14 Tsikl ‘for’ va ‘while’ operatori

while ifoda:
operator

birinchi ifoda tekshiriladi agar ifoda rost bulsa operator bajariladi va ifoda qayta hisoblaniladi.To ifoda 0 bulmaguncha sikl qaytariladi.
Agar dastur while True yoki while 1 deb sikl bersak bu sikl cheksiz davom etadi.
masalan n gacha sonlar yig’indisini hisoblang dasturini tuzamiz.

```
s = 0
i = 0
n = int(input('ni kiriting: '))

while i <= n:
    s = s + i
    i = i + 1
print(f'javob {s}') # n = 3 <<<>>> 6
-----------------------------------------
i = 1
n = 10

while i <= n:
    print(i)
```

```
    i = i + 1
# 1
2
3
4
5
6
7
8
9
10
-----------------------------------------------

i = 1
n = 10
while i <= n:
    print(i,end = ‘ ’)
    i = i + 1
# 1 2 3 4 5 6 7 8 9 10
-----------------------------------------------

s = "Hello world"
for i in s:
    print(i,end='')
# Hello world
-----------------------------------------------

s = "Hello world"
for i in s:
    print(f'"{i}"',end='')
# "H""e""l""l""o"" ""w""o""r""l""d"
-----------------------------------------------

s = "Hello world"
for i in s:
    if i == 'l':
        continue
    print(f'"{i}"',end='')

# "H""e""o"" ""w""o""r""d"
```

```
-----------------------------------------

for i in 'Helloworld':
    if i == ' ':
        break
    print(i,end='')
else:
    print("\nNo spaces")
# Helloworld
# No spaces
-----------------------------------------
```

15 Ro'yxat (List)

Pythonda ma'lumotlarni tartibli shakliga ro'yxat deyiladi.Ro'yxat massiv shakliga o'xshaydi lekin unda malumot turlari har xil bo'lishi mumkin masalan:

l2 = ['String', 'True', 14, '['start', 'stop', 'step']'] va hakozo ko'rinishda bo'lishi mumkin.
ro'yxat hosil qilishning ikkinchi usuli **list()** konstruktori yordamida amalga oshirish mumkin bunda kvadrat qavslar ishlatilmaydi

l3 = list(('String', 'True', 14, '['start', 'stop', 'step']'))
ro'yxat elementlariga murojat qilish quyidagicha bo'ladi.

```
print(l3[0])  #String
car = ['BMW', 'Ford', 'Audi']
print(car[-1]) #Audi ro'yxatga minus ishora bilan murojat qilish oxiridan boshlab hisoblab kelishga to'g'ri keladi

l2 = list('Hello')
print(l2)  #['H', 'e', 'l', 'l', 'o']
-----------------------------------------------------------

l2 = list(('Hello','World'))
print(l2)   #['Hello', 'World']
-----------------------------------------------------------

l5 = [i for i in 'Hello world' if i!=' ']
print(l5) #['H', 'e', 'l', 'l', 'o', 'w', 'o', 'r', 'l', 'd']
```

```
-----------------------------------------------------------
l6 = [i for i in 'Hello world' if i not in ['a','w','d',' ',’e’]]
print(l6)  # ['H', 'l', 'l', 'o', 'o', 'r', 'l']
-----------------------------------------------------------
```

range(diapazon) funksiyasi haqida

```
range(start,stop,step)
print(list(range(0,10))) #[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(list(range(0,10,2))) # [0, 2, 4, 6, 8]
-----------------------------------------------------------
for i in range(1,3):
    print(f"Tashqi sikl #{i}")
    for j in range(1,3):
        print(f'    Ichki sikl #{j}')
#Tashqi sikl #1
    Ichki sikl #1
    Ichki sikl #2
 Tashqi sikl #2
    Ichki sikl #1
    Ichki sikl #2
```

Ro’yxatda element mavjudligini tekshirish

```
fruits = ['Apple', 'Banana', 'Kiwi']
if 'Kiwi' in fruits:
    print('Yes')   # Yes
else:
    print('Not')
-------------------------------------------------
```

Element qiymatini o’zgartirish

```
fruits = ['Apple', 'Banana', 'Kiwi']
print(fruits)
fruits[0] = 'Orange'
```

```
print(fruits)
# ['Apple', 'Banana', 'Kiwi']
# ['Orange', 'Banana', 'Kiwi']
-----------------------------------------------
```

Uyga Vazifa Ko’paytirish jadvali

```
for i in range(1, 10):
    for j in range(1, 10):
        print(f'{i} * {j} = {i*j}')
    print('--------------')
print("Well Done !")
```

16 Ro’yxat metodlari

Yaratilgan ro’yxat ustida metodlar bajaramiz ro’yxat metodlarini ko’rish uchun quyidagi kod orqali bilishimiz mumkin.

```
x = dir(list)
print(x)
```

```
['__add__', '__class__', '__class_getitem__', '__contains__',
'__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattribute__', '__getitem__',
'__getstate__', '__gt__', '__hash__', '__iadd__', '__imul__',
'__init__', '__init_subclass__', '__iter__', '__le__', '__len__',
'__lt__', '__mul__', '__ne__', '__new__', '__reduce__',
'__reduce_ex__', '__repr__', '__reversed__', '__rmul__',
'__setattr__', '__setitem__', '__sizeof__', '__str__',
'__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend',
'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
```

shulardan asosiy metodlari

```
['append', 'clear', 'copy', 'count', 'extend', 'index', 'insert',
'pop', 'remove', 'reverse', 'sort']
```

List.append(x) - Ro`yxat oxiridan element qo`shish

List.extend(L) - Oxiriga hamma elementlarni qo`shib list ro`yxatini kengaytiradi.

List.insert(i,x) - i-elementga x qiymatini kiritadi

List.remove(x) - Ro`yxatdan x qiymatga ega elementni o`chiradi

List.pop([i]) - Ro`yxatning i-elementini o`chiradi va qaytaradi. Agarda indeks

ko`rsatilmagan bo`lsa oxirgi element o`chiriladi

List.index(x,[start],[end]) - X qiymatga teng start dan end gacha birinchi elementni qaytaradi

List.count(x) - X qiymatga teng elementlar sonini qaytaradi

List.sort([key=funksiya]) - Funksiya asosida ro`yxatni saralaydi

List.reverse() - Ro`yxatni ochadi

List.copy() - Ro`txatning nusxalaydi

List.clear() - Ro`yxatni tozalaydi

1.Element qo’shish

```
fruits = ['apple', 'banana', 'kiwi']

fruits.append('orange')

print(fruits)  #['apple', 'banana', 'kiwi', 'orange']
```

2.Agar elementni ro’yxatning malum qismiga qushmoqchi bulsa unda insert() metodini ishlatishimiz kerak.

```
fruits = ['apple', 'banana', 'kiwi']

fruits.insert(1, 'orange')

print(fruits) #['apple', 'orange', 'banana', 'kiwi']
```

3.Ro’yxat elementlarini o’chirish element nomi buyicha

```
fruits = ['apple', 'banana', 'kiwi']

fruits.remove('kiwi')

print(fruits)  #['apple', 'banana']
```

4.Ro’yxat indeksi buyicha elementni o’chirish.Agar indeks bulmasa oxiridan o’chiradi

```
fruits = ['apple', 'banana', 'kiwi']

fruits.pop(1)
```

```
print(fruits) #['apple', 'kiwi']
```

5.Ro’yxatni tozalash clear() funksiyasi orqali amalga oshirish mumkin.

```
fruits = ['apple', 'banana', 'kiwi']

fruits.clear()

print(fruits)  #[]
```

6.Ro’yxatni nusxalash ya’ni ro’yxatni ikkinchi o’zgaruvchi bilan nusxalash.

```
fruits = ['apple', 'banana', 'kiwi']

fruits2 = fruits.copy()

print(fruits2)    #['apple', 'banana', 'kiwi']
```

7.extend() funksiyasi bilan ro’yxatlarni kengaytirish mumkin va birinchi ro’yxatning oxiridan qo’shiladi.

```
fruits = ['apple', 'banana', 'kiwi']

fruits2 = ['apple', 'orange', 'pineapple']

fruits.extend(fruits2)

print(fruits)  #['apple', 'banana', 'kiwi', 'apple', 'orange',
'pineapple']
```

8.Ro’yxatdagi elementlarni sanash metodi count()

```
fruits = ['apple', 'banana', 'kiwi','apple', 'banana',
'kiwi','apple']

fruit = fruits.count('apple')

print(fruit)   #3
```

9. index() funksiyasi belgilangan elementning indeksini aniqlaydi. Agar bunday elementlar bir nechta bo’lsa, faqat birinchisining indeksini aniqlaydi.

```
fruits = ['apple', 'banana', 'kiwi','kiwi']

fruits = fruits.index('kiwi')

print(fruits) # 2
```

10. sort() funksiyasi ro’yxatni tartiblaydi. Agar ro’yxat sonlardan tashkil topgan bo’lsa, o’sish tartibida, satr yoki farflardan tashkil topgan bo’lsa, alifbo bo’yicha tartiblaydi.

```
fruits = ['banana', 'pineapple', 'kiwi','apple']
number = [1,3,5,2,9,7,4]
number.sort()
print(number)  # [1, 2, 3, 4, 5, 7, 9]
fruits.sort()
print(fruits)  # ['apple', 'banana', 'kiwi', 'pineapple']
```

11. reverse() funksiyasi ro’yxatning joriy holatdagi tartibini teskarisiga o’zgartiradi.

```
fruits = ['banana', 'pineapple', 'kiwi','apple']
number = [1,3,5,2,9,7,4]
number.reverse()
print(number)  # [4, 7, 9, 2, 5, 3, 1]
fruits.reverse()
print(fruits)  # ['apple', 'kiwi', 'pineapple', 'banana']
```

17.Kortejlar(Tuple) va ularning metodlari

Kortej elementlari xuddi to’plam elementlariga o’xshaydi.Ularni qiymatini o’zgartirib bo’lmaydi.Bittalik kortej hosil qilish uchun elementdan keyin vergul qo’yish kerak bo’ladi aks holda uni satr deb tushunadi

```
fruit = ('apple')
print(type(fruit))  # <class 'str'>
fruit = ('apple',)
print(type(fruit))  # <class 'tupple'>
```

Elementlarga murojat qilish uchun xuddi ro’yxatdagiday murojat qilinadi.

```
fruits = ('apple', 'banana', 'orange')
print(fruits[1])  # banana
```

Kortej metodlari xuddi list ro’yxat metodlariga o’xshaydi lekin ularni dir() funksiyasi orqali ko’rib bo’lmaydi

IKki kortejni qo’shish uchun quyidagi + amal orqali amalga oshirish mumkin

```
number1 = (1,3,4,5,6,7,2,1)
```

```
number2 = (8,9,1,3,4,5)

print(number1 + number2)#(1, 3, 4, 5, 6, 7, 2, 1, 8, 9, 1, 3, 4, 5)
```

Kortej(Tuple) hosil qilish uchun ikkinchi usuldan ham foydalanish ham mumkin.Uning uchun tuple() funksiyasidan foydalanamiz.

```
number = tuple((1,2,3,4,5,6,7))
```

18. To’plam(Set)

Pythonda to’plamlar bilan ishlash uchun maxsus **set** deb nomlanuvchi ro’yxat turi mavjud.

Pythondagi to`plam- **tasodifiy tartibda** va **takrorlanmaydigan** elementlardan tashkil topgan

“konteyner” deyiladi. To’plam elementlari tartiblanmagan va indekslanmagan tarzda bo’ladi.

To’plamni hosil qilish uchun maxsus qavslardan foydalaniladi. Yoki **set()** konstruktori ishlatiladi:

```
fruits = {'apple', 'banana', 'kiwi'}

print(fruits)     # {'kiwi', 'banana', 'apple'}
```

To’plamni hosil qilishning ikkinchi usuli

```
fruits = set(('apple', 'banana', 'orange'))

print(fruits)     # {'orange', 'banana', 'apple'}
```

19.To’plam metodlari

```
x = dir(set)

print(x)

#['__and__', '__class__', '__class_getitem__', '__contains__',
'__delattr__', '__dir__', '__doc__', '__eq__', '__format__',
'__ge__', '__getattribute__', '__getstate__', '__gt__', '__hash__',
'__iand__', '__init__', '__init_subclass__', '__ior__', '__isub__',
'__iter__', '__ixor__', '__le__', '__len__', '__lt__', '__ne__',
'__new__', '__or__', '__rand__', '__reduce__', '__reduce_ex__',
'__repr__', '__ror__', '__rsub__', '__rxor__', '__setattr__',
'__sizeof__', '__str__', '__sub__', '__subclasshook__', '__xor__',
'add', 'clear', 'copy', 'difference', 'difference_update',
'discard', 'intersection', 'intersection_update', 'isdisjoint',
'issubset', 'issuperset', 'pop', 'remove', 'symmetric_difference',
'symmetric_difference_update', 'union', 'update']
```

Bulardan Asosiy metodlarni ko’rib chiqamiz

1.intersection() - ikki to’plamdan ikkovida borini chiqaradi

```
print({1,2,3,4}.intersection({2,3,5,7}))      # {2, 3}
print({1,2,3,4} & {2,3,5,7})         # {2, 3}
```

2.union() ikki to’plamdan har birida borini bittadan chiqaradi

```
print({1, 2, 3, 4, 5}.union({3, 4, 5, 6})) #{1, 2, 3, 4, 5, 6}
print({1, 2, 3, 4, 5} | {3, 4, 5, 6})  ##{1, 2, 3, 4, 5, 6}
```

3.differense() ikki to’plamdan faqat birinchisida borini chiqaradi

```
print({1, 2, 3, 4, 6}.difference({2, 3, 5}))  #{1, 4, 6}
print({1, 2, 3, 4, 6} - {2, 3, 5})   #{1, 4, 6}
```

4. symmetric_difference()Ikki to’plamdan har birlarida yoqlarini chiqaradi

```
print({1, 2, 3, 4}.symmetric_difference({2, 3, 5}))    #{1, 4, 5}
print({1, 2, 3, 4}^{2, 3, 5})   #{1, 4, 5}
```

5. update() ikki to’plamni qo’shish

```
s = {1,2}
s.update({4,3})
print(s) #{1, 2, 3, 4}
```

6.difference_update() ikki to’plamdan faqat birinchisi ikkinchisida yo’q bo’lganini chiqaradi

```
a = {1,2,3,5}
b = {3,2,4,7}
```

```
a.difference_update(b)
print(a)
```

Method	*Operator*
a.intersection(b)	*a & b*
a.union(b)	*a|b*
a.difference(b)	*a - b*
a.symmetric_difference(b)	*a ^ b*
a.issubset(b)	*a <= b*
a.issuperset(b)	*a >= b*

20.Lug’at(Dictionary)

Pythondagi lug`atlar kalit bo`yicha kirishga ruxsat etuvchi erkin obyektlarning tartiblangan jamlanmasi. Ularni yana assotsiativli massivlar yoki hesh jadvallar deb nomlaydilar. Soddaroq qilib aytadigan bo`lsak lug’at xuddi manzillar kitobiga o’xshaydi, ya’ni biror insonning ismini bilgan holda uning manzili yoki u bilan bo’g’lanish ma’lumotlarini olish mumkin. Dictionary – tartiblanmagan, o’zgaruvchan va indeksli to’plam. Bu to’plamda kalit-

qiymat (key-value) tushunchasi mavjud, ya’ni maxsus kalit va ularga mos keluvchi qiymatlar juftlgidan tashkil topgan. Chap tarafda kalitlar, o’ng tomonda esa ularga mos keluvchi qiymatlar joylashgan bo’ladi. Buni hoir dictionary to’plamini hosil qilib bilib olamiz. Bu quyidagicha amalga oshiriladi:

```
d = {'key': 'value'}

print(d)      # {'key': 'value'}

print(d['key'])    #value


d = {k:v for k,v in [('key', 'value',)]}

print(d) # {'key': 'value'}

d = dict([('key', 'value')])    # {'key': 'value'}

d = dict(key='value')  # {'key': 'value'}
```

lug’atni qiymatini chiqarish uchun

```
car = {
   'name': 'BMW',
   'model': 'X7',
   'year': 2018
   }

print(car['name'])    #BMW

print(car.get('model'))     #X7

-------------------------------------

for x,y in car.items():
   print(x,y)


# name BMW

model X7
```

```
year 2018
```

Lug’atga kalit va qiymat kiritish ham mumkin.

```
car = {
    'name': 'BMW',
    'model': 'X7',
    'year': 2018
    }
car['color'] = 'black'
for i in car:
    print(f'{i}: {car[i]}')
```

21.Lug’at metodlari

Lug’at metodlarini ko’rish uchun har doimgidek quyidagi kod orqali aniqlab olishimiz mumkin.

```
x = dir(dict)
print(x)
```

```
# ['__class__', '__class_getitem__', '__contains__', '__delattr__',
'__delitem__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__getitem__', '__getstate__', '__gt__', '__hash__',
'__init__', '__init_subclass__', '__ior__', '__iter__', '__le__', '__len__',
'__lt__', '__ne__', '__new__', '__or__', '__reduce__', '__reduce_ex__',
'__repr__', '__reversed__', '__ror__', '__setattr__', '__setitem__',
'__sizeof__', '__str__', '__subclasshook__', 'clear', 'copy', 'fromkeys',
'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']
```

1.clear() lug'atni tozalaydi.

```
car = {'name': "BMW", 'model': 'X6', 'year': 2011}

car.clear()

print(car)        # {}
```

2.copy() lug'atni nusxalaydi.

```
car = {'name': "BMW", 'model': 'X6', 'year': 2011}

car2 = car.copy()

print(car2)       #{'name': 'BMW', 'model': 'X6', 'year': 2011}
```

3.fromkeys(seq,value) seq kalitli value qiymatli lug'at yaratadi.

```
car = {'BMW', 'Ford', 'MS-Benz'}

dictionary = dict.fromkeys((car), None)

print(dictionary)

#  {'Ford': None, 'BMW': None, 'MS-Benz': None}
```

4.get() lug'atni kalit so'zi bo'yicha chiqaradi

```
mydict = {'name': 'Ford'}

print(mydict.get('name'))          #Ford
```

5.items() kalit va qiymat buyicha lug'atni chop etadi.

```
car = {'name': 'Ford', 'year': 2011, 'model': 'Mustang'}
```

```
print(car.items())

# dict_items([('name', 'Ford'), ('year', 2011), ('model', 'Mustang')])
```

6.keys() lug’at kalitlarini qaytaradi.

```
car = {'name': 'Ford', 'year': 2011, 'model': 'Mustang'}

print(car.keys())        # dict_keys(['name', 'year', 'model'])
```

7.pop() kalit buyicha lug’at elementini o’chiradi agar kalit bo’lmasa xatolik chiqaradi.

```
car = {'name': 'Ford', 'year': 2011, 'model': 'Mustang'}

print(car)  #{'name': 'Ford', 'year': 2011, 'model': 'Mustang'}

car.pop('name')

print(car)  #{'year': 2011, 'model': 'Mustang'}
```

8.popitem() lug’at kalitini qiymati bilan birga o’chiradi va tuple ko’rinishiga olib keladi

```
car = {'name': 'BMW', 'year': 2011, 'model': 'X6'}

print(car) #{'name': 'BMW', 'year': 2011, 'model': 'X6'}

car.popitem()

print(car) #{'name': 'BMW', 'year': 2011}

print(car.popitem()) #('year', 2011)
```

9.setdefault(keys, value) lug’atga kalit va qiymat qaytarib qushadi

```
car = {'name': 'BMW', 'year': 2011}

res = car.setdefault('model', 'X6')

print(car) #{'name': 'BMW', 'year': 2011, 'model': 'X6'}
```

10.update(dict) ikkita lug’atni bir biriga qushadi agar ikkala lug’atda ham bir xil kalit busa ikkinchisi birinchisini yangilaydi

```
car = {'name': 'BMW', 'year': 2011}

car2 = {'name': 'Ford', 'year': 2013, 'model': 'Mustang'}

car.update(car2)
```

```
print(car) # {'name': 'Ford', 'year': 2013, 'model': 'Mustang'}
```

11.values() lug'atdagi qiymatni qaytaradi

```
car = {'name': 'BMW', 'year': 2011}

print(car.values())      # dict_values(['BMW', 2011])
```

22. Funksiya haqida tushuncha

Python dasturlash tilida funksiyani kiritish uchun **def** kalit so'zidan foydalanamiz va ikki nuqta (:) orqali funksiya ichiga kodlarimizni yozamiz.Funksiya ma'lum bir kodlarni o'z ichiga olgan blok hisoblanadi va uning o'z parametrlariga (argument) egadir.

Misol:

```
def hello():
    print("Hello, World!")

hello()              # Hello, World!
```

bu yerda def kalit so'zi orqali funksiyaga nom berdik va hello() qavs ichini bo'sh qoldirdik yani parametrsiz qoldirdik, qavs ichida xoxlagancha parameter kirgizishimiz mumkin lekin dasturlashda kod yozayotganimizda parametrlar soni 4tadan oshmagani afzal.

Endi funksiyaga parameter bergan holda chiqaramiz.

```
def hello(name):
    print("Hello, My name is " + name)
```

```
hello('John')          # Hello, My name is John
```

endi ma’lumotlarni ekranga chiqarishda ikkita holatni ko’rib chiqamiz

```
1. def get_sum(a, b):
    print(a + b)
get_sum(4, 3)              #7
print(get_sum(4, 3))       #7
                             None
2. def get_sum(a, b):
        return a + b
get_sum(3, 4)              #Hech narsa chiqmaydi
print(get_sum(3, 4))       #7
```

1- funksiya ichida print (chop etish) funksiyasi bulgani uchun faqat funksiya uziga murojat qilgani uchun ekranga chiqadi

2- funksiyada esa return kalit so’zi yani qaytarish uchun ishlatiladi va ekranga ma’lumotni chiqarish uchun print() funksiyasi orqali shu funksiyaning uziga murojat qilish kerak.

23.Funksiya haqida tushuncha

Registerni aniqlovchi funksiyani yaratamiz

```
def set_register(s):
    if ' ' in s:
        return s.upper()
    else:
        return s.lower()
print(set_register('Hello world'))          #HELLO WORLD
print(set_register('HELLOWORLD'))           #helloworld
```

funksiyaga qiymat (argument) kiritishda quyidagi ikki ko’rinishda bo’lishi mumkin

```
def get_sum(a,b,c,d):
    print(a+b+c+d)
```

```
get_sum(1, 2, 3, 4)                 #  10
get_sum(1, 2,c = 0, d = 5)          # 8
```

funksiyaga argumentlar kiritish usuli

```
def get_sum(*args):
    return sum(args)
print(get_sum(1, 2, 3, 4))          # 10
def func(*args):
    print(args)
func(1, 2, 3)      # (1, 2, 3)
def func(**kwargs):
    print(kwargs)
func(a = 1, b = 2, d = 3)       # {'a': 1, 'b': 2, 'd': 3}
```

ko’p argumentli funksiya

```
def func(a, b, *args, **kwargs):
    print(a)
    print(b)
    print(args)
    print(kwargs)
func(1, 2 , 3, 4, c = 'hello' , d = 'world')
#
1
2
(3, 4)
{'c': 'hello', 'd': 'world'}
```

24.Funksiya haqida tushuncha

Funksiya parametric sifatida listlarni tanlash

```
l = [1, 2, 3]

def func(lst):
    return [i * 2 for i in lst]

print(func(l)) #[2, 4, 6]
```

funksiya ichida funksiya

```
def func(lst):
    def get_mult(i):
        return i * 2
    return [get_mult(j) for j in lst]

print(func([1, 2, 3]))    #[2, 4, 6]
```

lokal va global o’zgaruvchilarni funksiya ichiga kiritish

```
a = 5
def func():
    global a
    a = 10
    return a
print(a) # 5
print(func()) # 10
print(a) # 10
```

list ichini tekshirish

```
def isInstance(lst):
    def get_mult(x):
        if isinstance(x, int):
            return x
    return [get_mult(i) for i in lst if isinstance(i, int)]
print(isInstance([1, '2', 3, ‘4’, 5, ‘6’]))  #[1, 3, 5]
```

map() metodi yordamida funksiya natijasini o’zgartirish

```
l = [1, '2', 3]
def f4(lst):
    def get_mult(x):
        return x * 2
    return list(map(get_mult, l))
print(f4(l))  # [1, ‘22’, 3]
```

25.join(), map(), filter() metodlari

join() metodi haqida:

Umumiy ko'rinishi ***string_name.join(iterable)*** bu metod orqali satrlarni elementlari orasiga ma'lumot kiritadi.

iterable – bir vaqtning o'zida o'z a'zolarini qaytara oladigan obyekt bunga List, Tuple, String, Dictionary va Set lar kiradi.

```
list1 = ['h', 'e', 'l', 'l', 'o']

print("".join(list1))     # hello


list1 = " hello "

print("$".join(list1))  # $h$e$l$l$o$
```

```
dic = {'Hello': 1, 'World': 2}
string = '_'.join(dic)
print(string)     #Hello_World
```

join metodini split metodi bilan birga ishlatish

```
s = 'hello my world'
d = s.split(' ')
print(d)                # ['hello', 'my', 'world']
print('-'.join(d))      # hello-my-world
```

map() metodi:

Umumiy ko'rinishi ***map(fun, iter)***

```
def addition(n):
    return n + n
numbers = (1, 2, 3, 4)
result = map(addition, numbers)
print(list(result))    #[2, 4, 6, 8]
```

lambda funksiyasi orqali ifodalash bu sulda kodimiz qisqaroq buladi

```
numbers = (1, 2, 3, 4)
result = map(lambda x: x + x, numbers)
print(list(result))    #[2, 4, 6, 8]
```

list ichidagi stringlarni list xolatiga keltirish

```
list1 = ['h', 'e', 'l', 'l', 'o']
test = list(map(list,list1))
print(test)   #[['h'], ['e'], ['l'], ['l'], ['o']]
```

filter() metodi:

Umumiy ko’rinishi ***filter(function, sequence(iterable))***

misol: Ikkala ismda harflarni tekshiruvchi dastur tuzish

```
def fun_name(var):
    name = 'Shaxboz'
    if var in name:
        return True
    else:
        return False
name1 = 'Shaxzod'
filtered = filter(fun_name, [i for i in name1])
print('ikkala ismda harflar bor: ')
for s in filtered:
    print(s)
#ikkala ismda harflar bor:
S
h
a
x
z
o
```

2-misol:to’plamdan juft va toq raqamlarni aniqlash dasturini tuzish

```
numbers = [1,2,3,4,5,6,7,8,9]

juft = filter(lambda x: x % 2 == 0, numbers)
print(list(juft))       # [2, 4, 6, 8]

toq = filter(lambda x: x % 2 != 0, numbers)
print(list(toq))        # [1, 3, 5, 7, 9]
```

26.Pythonda modular

Pythonda ayrim narsalarga mo’ljallangan tayyor, maxsus modular bor.Ularning har birini o’z vazifasi bor.Modullar kodlashni osonlashtirish uchun qo’llaniladi va bir qancha modullarni o’rganamiz va o’zimiz yaratgan modullarni ishlatishni o’rganamiz

1.math moduli:

1.1 ceil(x)

```
import math
m = math.ceil(33.8)
print(m)  #34
```

1.2 floor(x)

```
import math
m = math.floor(33.8)
print(m)  #33
```

1.3 import math

```
print(math.e)     # 2.718281828459045
```

```
1.4 import math
print(math.pi)    # 3.141592653589793
1.5 factorial(x)
import math
print(math.factorial(5))      # 120
1.6 fabs(-x)
import math
print(math.fabs(-10))   # 10
1.7 pow(a, b)
import math
print(math.pow(2, 4))         # 16.0
print(math.pow(8, 0.5))       #2.8284271247461903
```

Trigonometrik funksiyalar uyga vazifa----->>>>>>

2.random moduli:

random moduli tasodiflarni chiqaruvchi modul hisoblanadi

```
import random
print(random.random())        # 0.902855379558181
print(random.randrange(1, 100))     # 34
```

random modulini bir qancha funksiyalari mavjud:

- random() 0.0 va 0.1 oraliqda butun sonlar generatsiyasi
- choice(x) x ketma ketlikdan tasodifiy elementni generatsiyalash
- shuffle(x) x ketma ketlikni barcha elementlarini generatsiyalaydi
- randrange(start, stop, step) oraliqdagi raqamlarni qadamma qadam generatsiyalaydi

choice() funksiyasi

```
import random
x = [1,2,3,4,5,6]
print(random.choice(x)) #4
```

shuffle() funksiyasi

```
import random
x = [1,2,3,4,5,6]
random.shuffle(x)
```

```
print(x) # [3, 1, 5, 2, 6, 4]
```

randrange() funksiyasi

```
import random
print(random.randrange(1, 100, 1)) # 44
```

Pythonda o'zimizni modulni yaratamiz:

Birinchi module faylimizni yaratamiz

C:\Users\rootSystem\Documents\mymodule.py

```
def get_sum(*args):
    return sum(args)
```

C:\Users\rootSystem\Documents\print.py

```
import mymodule
print(mymodule.get_sum(1,2,3))     # 6
```

Bu kodimizni qisqaroq shaklda ham kiritishimiz mumkin

C:\Users\rootSystem\Documents\print.py

```
import mymodule as my
print(my.get_sum(1,2,3))     # 6
```

yoki

```
from mymodule import get_sum
print(get_sum(1,2,3))   # 6
```

va yoki

```
from mymodule import get_sum as gs
print(gs(1,2,3))  # 6
```

27.DateTime moduli

python dasturlash tilida datetime moduli sana va vaqt ustida ishlash uchun yordam beruvchi modul hisoblanadi.

```
import datetime as date

today = date.datetime.now()

print(today)      #  2022-03-02 20:11:59.456700
```

datetime moduli orqali biz vaqt yoki sanani o’zimizga kerakli qismlarini ham olishimiz mumkin masalan:

```
import datetime as date

dt = date.datetime.now()

print(dt.month)      # 3

print(dt.day)        # 2

print(dt.year)       # 2023

print(dt.weekday())  #3
```

*Bular hammasi strftime() funksiyasi ichiga kiritilgan xolatda ishlatiladi

- **%a – hafta kuni (qisqa)**
- **%A – hafta kuni (to'liq)**
- **%w – hafta kuni (raqam shaklida)**
- **%d – oyning sanasi**
- **%b – oy nomi (qisqa)**
- **%B – oy nomi (to'liq)**
- **%m – oy (raqam ko'rinishida)**
- **%y – yil (qisqa)**
- **%Y – yil (to'liq)**
- **%H – soat (00-23)**
- **%I – soat (00-12)**
- **%p – kun vaqti (AM/PM)**
- **%M – minut (00-59)**
- **%S – sekund (00-59)**
- **%j – yildagi kun raqami (001-366)**
- **%U – yildagi hafta raqami, Yakshanba birinchi kun sifatida (00-53)**
- **%W – yildagi hafta raqami, Dushanba birinchi kun sifatida (00-53)**
- **%c – mahalliy sana va vaqt**
- **%x – mahalliy sana**
- **%X – mahalliy vaqt**

datetime moduli orqali o'zimiz hohlagan sana va vaqtni kiritish mumkin

```
import datetime
enter_date = datetime.date(2022, 5, 20)
print(enter_date)    # 2022-05-20
```

1.datetime obyektidan vaqt va sanalarni chiqarish

```
import datetime
a = datetime.datetime.now()
print(a)        # 2023-03-02 20:46:16.970869
# Sanani olish
b = a.date()
```

```
print(b)        # 2023-03-02
# Vaqtni olish
c = a.time()
print(c)        # 20:46:16.970869
```

2.Uyga vazifa: ikki sana orasidagi kunni hisoblash dasturi

```
import datetime
date1 = datetime.date(2022, 5, 20)
date2 = datetime.date(2022, 7, 20)
print(date2 - date1)          # 61 days, 0:00:00
```

3.Sana, oy va kunlarni o'zgartirish

```
from datetime import date
d1 = date(2019, 5, 20)
d2 = d1.replace(year=2022, month=7, day=23)
print(d1, d2, sep='\n')
# 2019-05-20
2022-07-23
```

4.Hozirgi sanaga kun qo'shish

```
import datetime
today = datetime.datetime.today()
print(today)  # 2023-03-02 21:12:52.529487
delta = datetime.timedelta(24)
print(delta)   # 24 days, 0:00:00
result = today + delta
print(result)  # 2023-03-26 21:12:52.529487
```

5.Sanani har xil formatda ekranga chiqarish

```
from datetime import date


a = date.today()
```

```
print(a)
print(a.strftime('%d.%m.%Y'))
print(a.strftime(f'Bugun sana %d - %B. %Y - yil ')
#
2023-03-02
02.03.2023
Bugun sana 02 - March. 2023 – yil
----------------------------------------
import locale
import datetime


a = locale.setlocale(locale.LC_ALL, 'uz-Uz.UTF-8')
b = datetime.datetime.today()
print(b.strftime("%A").capitalize()) #Juma
# print(a)
```

28.Pythonda fayllar ustida ishlash

Pythonda fayllar bilan ishlash juda muhim hisoblanib, fayllarni ochish, hosil qilish, o’zgartirish va boshqa amallarni qilish mumkin.

Pythonda fayllarni ochish uchun open funksiyasi ishlatiladi va bu funksiya ikkta argumentdan iborat

f = open(filename, mode)

‘r’ – faylni o’qish uchun ochish,agarda fayl mavjud bo’lmasa xatolik yuz beradi.

‘a’ – faylni o’zgartirish va qo’shimcha kiritish uchun ochadi,agar fayl bo’lmasa fayl yaratadi.

‘w’ – faylga yozish uchun ochish,agar fayl bo’lmasa fayl yaratadi.

‘x’ – yangi fayl hosil qilish,agarada bunaqa fayl bo’lsa xatolik yuz beradi

‘t’ – fayl matndan iborat

‘b’ – fayl binary matndan iborat

‘r+’ – faylni o’qish va yozish rejimi

‘w+’ – yozish va o’qish rejimi

‘a+’ – faylga ma’lumot qo’shish va o’qish rejimi

Fayl hosil qilish

```
f = open('new_file.txt', 'x')
f.close()
```

hozir new_file.txt formatda fayl hosil bo’ldi

Faylga yozish

```
f = open('new_file.txt', 'w')
f.write('Hello, world')
f.close()
```

Faylni o’qish

```
f = open(‘new_file’, ‘r’)
print(f.read())   # Hello, world
```

Faylga qo'shimcha kiritish uchun ochish

```
f = open('new_file.txt', 'a')
f.writelines('\nPineApple')
f.close()
```

```
Apple
Banana
Kiwi
PineApple
```

Faylni qatorma qator o’qish

new_file.txt

```
Apple
Banana
Kiwi
PineApple
```

print.py

```
file_name = 'new_file.txt'
with open(file_name, 'r') as text_file:
    for lines in text_file:
```

```
        print(lines)
```

```
Apple


Banana


Kiwi


PineApple
```

Faylga listlardagi ma’lumotni kirish

```
f = open('new_file.txt', 'w')
fruits = ['apple', 'banana', 'kiwi']
for fruit in fruits:
    f.write(fruit + '\n')
f.close()
```

Uyga vazifa ('\n') larni yo'qotgan holda list hosil qilish dasturini tuzish

```
fruits = []

for i in open('new_file.txt'):
    fruits.append(i)

print(fruits)

for i in range(len(fruits)):
    if fruits[i][-1] == '\n':
        fruits[i] = fruits[i][:-1]

print(fruits)
```

```
# ['apple\n', 'banana\n', 'kiwi\n']
# ['apple', 'banana', 'kiwi']
```

Undan tashqari

```
fruist = ['apple', 'banana', 'kiwi']
f = open('new_file.txt', 'a')
f.writelines(fruist)
f.close()
```

 # applebananakiwi

```
fruist = ['apple', 'banana', 'kiwi']
f = open('new_file.txt', 'a')
f.writelines(f'{fruit}\n' for fruit in fruist)
f.close()
```

apple

banana

kiwi

29.Html Parsing

1.Pypi.org saytidan bizga kerakli bo'lgan paket menejerlarni yuklab olamiz,bizga paketi hamda urllib standatr paket to'plami kerak bo'ladi

```
from bs4 import BeautifulSoup
import urllib.request
```

2.Keyin esa xoxlagan saytga http so'rov yani request jo'natamiz va html kodini ko'ramiz

```
req = urllib.request.urlopen('https://zamin.uz/uz/')
# print(req)
html = req.read()
# print(html)
```

3.BeautifulSoup4 paket menejeri orqali uni tartibli html kodlarga keltiramiz yani parser holatiga olib kelamiz

```
soup = BeautifulSoup(html, 'html.parser')
# print(soup)
```

4.Endi saytni o'zimizga kerakli bo'lgan joylarini parserlaymiz

```
news = soup.find_all('div', class_='short-item')
```

```
# print(news)
```

5.Usha kerakli joylarni list ko’rinishida bo’lgani uchun alohida alohida list elementlariga o’tkazamiz for tsikl operatoridan foydalangan holda

```
results = []

for item in news:
    news_title = item.find('div', class_='short-text').get_text(strip=True)
    # print(news_title)
    news_date = item.find('div', class_='short-date fx-1 nowrap').get_text()
    # print(news_date)
    news_url = item.a.get('href')
    # print(news_url)
    results.append({
        'news_title': news_title,
        'news_date': news_date,
        'news_url': news_url
    })
```

6.Endi saytdan olgan ma’lumotlarni faylga saqlab qo’yamiz

```
f = open('parsing-zamin.uz', 'w', encoding='utf-8')
j = 1
for i in results:
    f.write(f'News № {j}\nNews title: {i["news_title"]}\nNews date:
{i["news_date"]}\nNews url: {i["news_url"]}\n--------------\n')
    j = j + 1

f.close()
```

30.Xatolar va istisnolar

Pythonda yozgan kodimiz xatolik yoki biror bir istisno holatlar bo’lib qolsa, python bizga xabar beradi.Bunday istisno holatlar bilan ishlash uchun try va except kalit so’zlari (bloklaridan) foydalaniladi.

```
print(100/0)  #ZeroDivisionError: division by zero
print(1 + '2')  #TypeError: unsupported operand type(s) for +: 'int' and 'str'
print(int('test'))      #ValueError: invalid literal for int() with base 10: 'test'


try:
   num = 100/0
except ZeroDivisionError:------>>>> Maxraj nolga teng bo'lmagan qiymat
   num = 0
print(num)    # 0
```

```
try:
    num = 100/2
except ZeroDivisionError:
    num = 0

print(num)     # 50.0

try:
    num = 100/'2'
except Exception: ------>>>> Xoxlagan turlardagi istisno uchun
    num = 0

print(num)     # 0
```

else kalit so'zi orqali kodda hech qanday xatolik bo'lmagan vaqtda ishlatiladi.

```
try:
    print('Salom')
except:
    print('Dasturda xatolik bor')
else:
    print('Dasturda xatolik yuq')
```

Salom

Dasturda xatolik yuq

finally kalit so'zi esa dasturda kod bo'ladimi bo'lmaydimi amal bajariladi va datur xatoligi haqida axborot beradi

```
try:
    print(100/'2')
except:
```

```
    print('100'/2)
finally:
    print('''Dasturada hecham xatolik yuq
Lekin pastda xatoliklar baribir chiqadi''')
# Dasturada hecham xatolik yuq
Lekin pastda xatoliklar baribir chiqadi
Traceback (most recent call last):
  File "C:/Users/rootSystem/Documents/print.py", line 2, in <module>
    print(100/'2')
TypeError: unsupported operand type(s) for /: 'int' and 'str'
During handling of the above exception, another exception occurred:
Traceback (most recent call last):
  File "C:/Users/rootSystem/Documents/print.py", line 4, in <module>
    print('100'/2)
TypeError: unsuppor ted operand type(s) for /: 'str' and 'int'
```

Misol:Faqat butun son kiritish dasturini tuzish

```
while True:
        try:
                nb = int(input('Enter a number: '))
                break
        except ValueError:
                print('This is not a number, try again.')
```

Input: Enter a number: 5

Input: Enter a number: r

Output: This is not a number , try again

Output: Enter a number:

Undan tashqari istisno holati ham mavjud istisno holatini o'zimiz balki interpretator ko'rsatmaydi masalan butun son kiritish haqida istisno tuzamiz agar qiymat butun son bo'lmasa xatolik haqida o'zimiz xabar beramiz

```
int_number = 'Hello'

if type(int_number) is not int:

   raise TypeError('Kiritilgan son butun son emas')

# TypeError: Kiritilgan son butun son emas
```

Kiritilgan malumotni aniqlab kurish dasturi

```
int_number = input('Enter yor number: ')


print(type(int_number))


if type(int_number) is not int:

   raise TypeError('Kiritilgan son butun son emas')
```

31.Asosiy OOP asoslari.Klasslar va obyektlar haqida tushuncha

Object-oriented programming (**OOP**) yani obyektga yo'naltirilga dasturlash degan ma'noni beradi va bunday dasturlash tillari dinamik dasturlash tillari deb yuritiladi.O'zi umuman dasturlash tillari ikki xil bo'ladi.

- Strukturali dasturlash
- Dinamik dasturlash

Strukturali dasturlash tillariga misol qilib C dasturlash tilini olsak bo'ladi yani Strukturali dasturlash tillarida bilan dinamik dasturlash tillari orasidagi asosiy farq strukturali dasturlash tilida classlar bilan ishlash imkoniyati yo'q hisoblanadi.Dinamik dasturalsh tillari orqali esa obyekt va sinflar bilan ishlash imkoni bor va dinamik dasturlash tillari asosan strukturali dasturlash tillari orqali yaratiladi masalan hozirda eng mashhur dasturlash tillaridan deyarli barchasi C dasturlash tili orqali tuzulgan (Python,C++,Java,JavaScript,Php va hakozo)

Python dasturlash tili ham obyektga yo'naltirilgan dasturlash tillari safiga kiradi hamda biz bu dasturlash tili orqali osongina obyektlar bilan ishlash imkonini beradi.Obyektga yo'naltirilgan dasturlash tillari asosiy prinsplari quyidagilar kiradi.

- object
- class
- method
- inheritance
- polymorphism
- data abstraction
- encapsulation

Misol:

```
class A:
    pass
a = A()
a.property = 'Property'
print(a.property)      # Property
```

```
class Hi:
    def say_hi(self):
        return 'Hi'
a = Hi()
print(a.say_hi())      # Hi
```

```
class A:
    prop1 = 'Property1'
    prop2 = 'Property2'
a = A()
print(a.prop1)          # Property1
```

Bu yerda A nomli sinf hosil qilindi hamda a nomli esa obyekt hosil bo'lyapti.

self parametri bu usha sinfga tegishli ekanligini bildiradi va usha sinfdan chetga chiqib ketmaydi.Misol:

```
class Hello:
    def hello_name(self, name):
        return 'Hello ' + name
hi = Hello()
print(hi.hello_name('John'))         # Hello John
```

Obyektlarga parameter kiritish uchun yani xususiyat kiritish uchun __init__ funksiyasidan foydalanishimiz kerak

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
person = Person("John", 23)
print(person.name)   # John
print(person.age)     # 23
```

Obyekt ichiga hohlagan funksiya kiritganimizda usha funksiyaga parametr kiritishimiz shart emas chunki usha __init__ funksiyasiga kiritilgan parametrlar usha obyekt ichidagi barcha funksiya uchun tegishli bo'ladi

Misol:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def say(self):
        return (f'My names is {self.name}, I am {self.age}')
```

```
person = Person("John", 23)
print(person.say())   # My names is John, I am 23
```

32.Obyekt xususiyatlarini o’zgartirish.Kostruktor tushunchasi

Dastlab tuzgan obyektimizni xususiyatlarini __init__ funksiyasi orqali kiritgan edik.Shunday qilib bu obyektimizni xususiyatlarini o’zgartirishimiz mumkin.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
    def say(self):
        return (f'My names is {self.name}, I am {self.age}')


person = Person('John', 24)
print(person.say())
print(person.age)
person.age = 32
print(person.age)
person.name = 'Katy'
print(person.say())
#My names is John, I am 24
#24
#32
#My names is Katy, I am 32
```

Obyekt xususiyatlari va obyekt o'zini o'chirish uchun del kalit so'zidan foydalanamiz

del person.age va yoki obyektni o'zini o'chirish uchun esa del person kabi bo'ladi

kiritayotgan obyektlarimizni biror boshqa fayl orqali ham chaqirib olishimiz mumkin buning uchun quyidagicha qilish kerak.

my_class.py

```
class Person:
    name = 'John'
    def print_info(self):
        print('Hello ' + self.name)
```

print.py

```
from my_class import Person

person = Person()

person.print_info()    #Hello John

person.name = 'Katy'

person.print_info()    #Hello Katy
```

Parametrlangan Konstruktor

```
class Addition:
        first = 0
        second = 0
        answer = 0
        def __init__(self, f, s):
                self.first = f
                self.second = s
        def display(self):
                print("First number = " + str(self.first))
                print("Second number = " + str(self.second))
                print("Addition of two numbers = " + str(self.answer))
        def calculate(self):
                self.answer = self.first + self.second
obj1 = Addition(1000, 2000)
obj2 = Addition(10, 20)
obj1.calculate()
obj2.calculate()
```

```
obj1.display()

obj2.display()


# First number = 1000

Second number = 2000

Addition of two numbers = 3000

First number = 10

Second number = 20

Addition of two numbers = 30
```

33.Vorislik

Vorislik tushunchasi – bir ona sinf va bir voris sinf hosil qilamiz va voris sinfga ona sinfni parametr sifatida kiritamiz.Shunda voris sin ona sinfni barcha xususiyatlarini (__init__) o’zida aks etadi.

Misol: Bu yerda Person nomli sinf ona sinf Student nomli sinf esa voris sinf hisoblanadi

```
class Person:

    def __init__(self, name, age):

        self.name = name

        self.age = age
```

```
    def say(self):
        print(self.name, self.age)
class Student(Person):
    pass
pr = Student('John', 23)
pr.say()            # John 23
```

super() metodi bu ona sinfdagi funksiya yoki parametrlarni voris sinfga o'zlashtirish uchun qo'llaniladi.

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
class Student(Person):
    def __init__(self, name, age, id):
        super().__init__(name,age)
        self.id = id
student = Student('John Doe', 23, 1)
print(f'Student id: {student.id}\nStudent name: {student.name}\nStudent age: {student.age}')
# Student id: 1
# Student name: John Doe
# Student age: 23
```

Uyga vazifa: Obyektlarda inkapsulatsiya

```
class B:
```

```
    def fun(self):
        print('Public method')
    def __fun(self):
        print('Private method')
objekt = B()
objekt._B__fun()
objekt.fun()
```

Private method

Public method

34. Polimarfizm

Polimorfizm dasturlashda juda muhim tushuncha bo'lib u kodimizni qisqaroq va tushunarli bo'lishini ta'minlaydi.U turli xil holatda har xil turlarni ifodalash uchun bitta obyektdan foydalanish imkonini beradi.

get.py

```
class Rectangle:
    def __init__(self, a, b):
        self.a = a
```

```
        self.b = b
    def __str__(self):
        return f'Rectangle {self.a} x {self.b}'
    def get_s(self):
        return self.a * self.b
class Square:
    def __init__(self, a):
        self.a = a
    def __str__(self):
        return f'Square {self.a}'
    def get_s(self):
        return self.a ** 2
class Circle:
    def __init__(self, r):
        self.r = r
    def __str__(self):
        return f'Circle {self.r}'
    def get_s(self):
        return (self.r ** 2) * 3.14
```

class_get.py

```
from get import Rectangle, Square, Circle


rec1 = Rectangle(4, 3)
rec2 = Rectangle(12, 5)
```

```
squ1 = Square(4)

squ2 = Square(7)


cir1 = Circle(2)

cir2 = Circle(4)


figures = [rec1, rec2, squ1, squ2, cir1, cir2]

for figure in figures:

   print(figure, '==>', figure.get_s())

Rectangle 4 x 3 ==> 12

Rectangle 12 x 5 ==> 60

Square 4 ==> 16

Square 7 ==> 49

Circle 2 ==> 12.56

Circle 4 ==> 50.24
```

35.Dekoratorlar

Bir funksiya yoki obyektni bir necha joyda ishlatishga to’g’ri kelib qolganda decoratorlardan foydalanish mumkin.Dekoratorni biz umumiy funksiya deb tushunishimiz mumkin va u funksiyalarni qiymatlarini o’zgartirish xususiyatiga ham ega.

Misol:

```
def decor(func):
    def wrap():
        print('=====')
        func()
```

```
        print('=====')
    return wrap
def print_txt():
    print('Hello decor func')


dec = decor(print_txt)
dec()    # Hello decor func
```

decorator orqali ifodalash

```
def decor(func):
    def wrap():
        print('=====')
        func()
        print('=====')
    return wrap
@decor
def print_txt():
    print('Hello decor func')
```

Misol: har xil turdagi Mevalarni chiqarish uchun quyidagi misolni ko’rib chiqamiz

```
def fruits(fruit):
   def fruit_name():
      name = fruit.__name__
      print(name)
   return fruit_name
@fruits
def apple():
```

```
    pass

apple() #apple
```

keyinchalik bu funksiyaga xususiyat qo'shishimiz mumkin

```
def fruits(fruit):
    def fruit_name():
        name = fruit.__name__
        if name == 'apple':
            color = 'red'
            print(f'{name} color {color}')
        elif name == 'banana':
            color = 'yellow'
            print(f'{name} color {color}')
        else:
            print(None)
    return fruit_name
@fruits
def apple():
    pass
apple()
```

36.Klass Parsing

```
from bs4 import BeautifulSoup
import urllib.request


class Parsing:

    def __init__(self, url):
        self.url = url

    def request_url(self):
        req = urllib.request.urlopen(self.url)
        html = req.read()
        soup = BeautifulSoup(html, 'html.parser')
        news = soup.find_all('div', class_='short-item')
        return news

    def create_parsing(self):
        results = []
        for item in self.request_url():
```

```
            news_title = item.find('div', class_='short-text').get_text(strip=True)

            news_date = item.find('div', class_='short-date fx-1 nowrap').get_text()

            news_url = item.a.get('href')

            results.append({
                'news_title': news_title,
                'news_date': news_date,
                'news_url': news_url
            })
        return results

    def write_parsing(self):
        f = open('parsing-zamin', 'w', encoding='utf-8')
        j = 1
        for i in self.create_parsing():
            f.write(
                f'News № {j}\nNews title: {i["news_title"]}\nNews date:
{i["news_date"]}\nNews url: {i["news_url"]}\n'
                f'--------------\n')
            j = j + 1

        f.close()


parsing = Parsing('https://zamin.uz/uz/')
parsing.write_parsing()
```

38. Modul SQlite. 1 – qism

Pythonda ma’lumotlar bazasi bilan ishlaganda turli xil modullarni ko’rib chiqishimiz mumkin.Misol uchun sqlite3 modulini ko’rib chiqamiz

```
# birinchi sqlite3 modulini qo’shib olamiz

import sqlite3

# xoxlagan biror o’zgaruvchi bilan .db yoki .sqlite kengaytmasidagi faylga ya’ni baza ulaymiz agar unaqa turdagi baza bo’lmasa baza hosil qiladi.

db = sqlite3.connect('test_db.sqlite')

# cursor() funksiyasi orqali bazani ulaymiz
```

```
cursor = db.cursor()
# execute() funksiyasi orqali bazamizni ichiga tablitsa yaratamiz
cursor.execute('''Create Table users (
id Integer Primary Key,
name Text Not Null,
email Text Not Null Unique
)''')
# close() funksiyasi orqali bazani yopamiz
db.close()
```

agar bu kodni qaytadan kompilatsiya qilsak xatolik chiqaradi